

Angular 21 Funding Panel Application

A junior-friendly knowledge transfer for the Sec Corp and PBIL panels

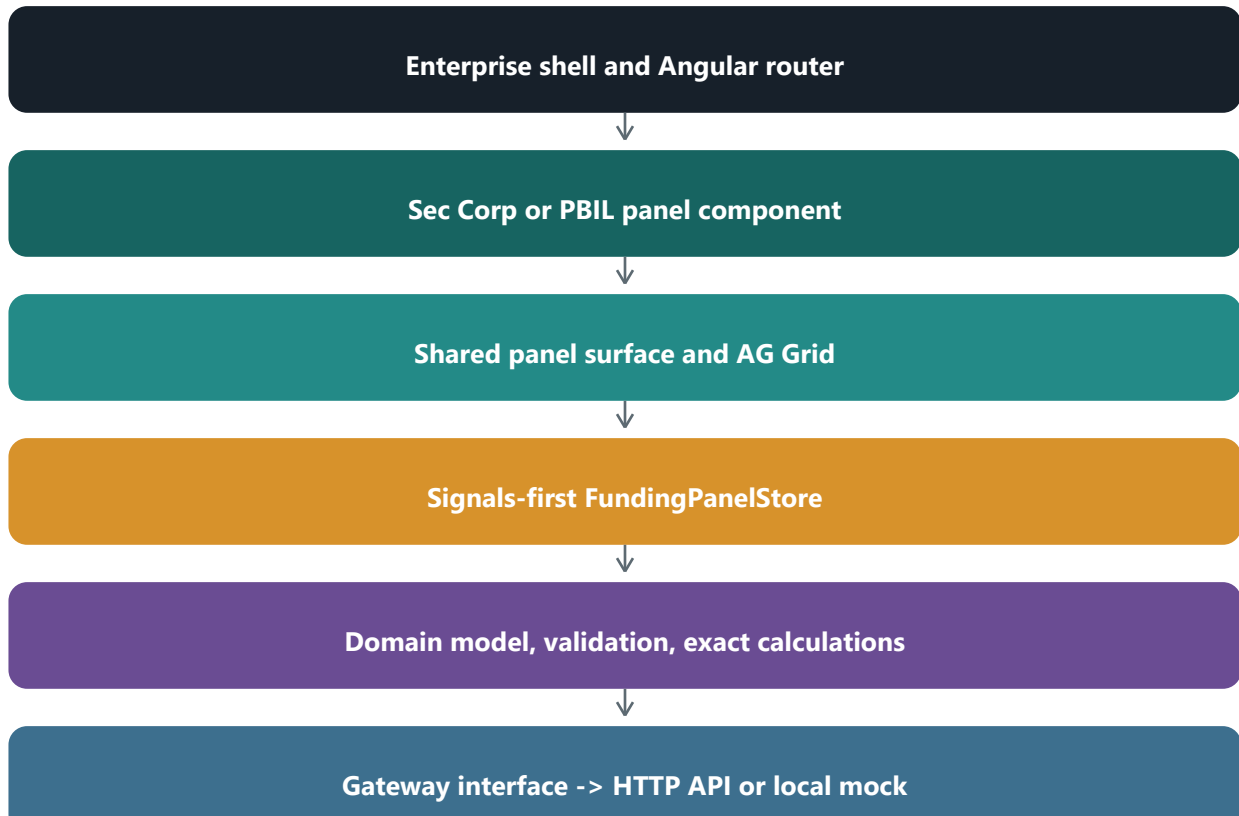
Purpose. Explain how the current production-oriented implementation is structured, how information moves through it, how editing and persistence work, why the major decisions were made, and where the design can evolve.

System boundary	Current implementation
Framework	Angular 21, standalone components, strict TypeScript
Panels	Sec Corp and PBIL
Grid	AG Grid 36 with runtime Enterprise license support
State	Angular signals, computed signals, narrowly scoped effects
Persistence	Versioned GET and full-state PUT through a gateway
Financial arithmetic	Canonical decimal strings and decimal.js
Quality gate	Prettier, ESLint, 77 tests, production build

Prepared from the implemented repository state. The enterprise shell remains responsible for docking, window chrome, dimensions, authentication context, and deployment-time configuration.

1. Start with the big picture

Think of the application as a reusable funding-panel engine. Sec Corp and PBIL are different reports, but they share the same machinery for loading, validation, calculation, editing, saving, concurrency, rendering, and shell communication.



The key separation: panel-specific facts live near each panel; reusable behavior lives in the shared funding-panel feature. A new report should normally add a route, a thin panel component, a row catalog, and data. It should not copy the store or grid.

What the application owns

- The report grid and its styling.
- Financial entry, validation, preview calculations, dirty state, and autosave.
- GET and PUT calls, runtime response validation, and optimistic concurrency.
- Loading, saving, error, and conflict messages.

What the enterprise shell owns

- Docking, resizing, minimize/maximize/close controls, and outer title chrome.
- Authenticated session and authorization context.
- Runtime business date, API base URL, and AG Grid Enterprise license key.
- Manual refresh requests and consumption of panel dirty/save status.

2. Component tree and folder structure

```
AppComponent
  RouterOutlet
    SecCorpPanelComponent OR PhilPanelComponent
      FundingPanelSurfaceComponent
        FundingGridComponent
          AgGridAngular
            FundingAmountCellEditorComponent
```

The panel components are deliberately thin. Their main job is to create an isolated dependency-injection scope and supply the correct panel code and mock fixture.

Folder	Responsibility
core/config	Reads deployment configuration and exposes it through dependency injection.
core/grid	Registers only the AG Grid modules used by this feature and sets the license.
core/host	Defines the small adapter used to communicate dirty, save, and refresh state.
funding-panel/domain	Pure business types, exact decimal rules, calculations, and invariants.
funding-panel/data-access	REST schemas, DTO mapping, gateway abstraction, HTTP adapter, and mock adapter.
funding-panel/application	Signals store and financial input validation. Orchestrates use cases.
funding-panel/presentation	View-model mapping, AG Grid, financial editor, styling, and status surface.
funding-panel/panels	Sec Corp and PBIL routes, row catalogs, fixtures, and panel-specific tests.

Why feature-first organization?

A layer-only project often grows into global folders such as components, services, and models. Developers then jump around the repository to understand one feature. Here, everything related to funding panels is together, while layers inside that feature still enforce clear dependency boundaries.

Tradeoff: feature-first organization can produce some repeated folder names across features. That is a small cost for stronger ownership and easier removal, testing, and reuse of a complete feature.

3. Read flow: API to pixels

The application follows this sequence when a panel opens:

Step	What happens
1	The router lazy-loads the Sec Corp or PBIL standalone component.
2	The panel creates its own FundingPanelStore and gateway provider scope.
3	FundingPanelSurfaceComponent calls store.load(panelCode, businessDate).
4	The selected gateway performs GET, using HTTP or the local mock.
5	The response is parsed as unknown, checked by Zod, mapped, and domain-validated.
6	The store saves the validated report as serverReport and computes a view model.
7	FundingGridComponent builds columns and gives rows to AG Grid.
8	Central formatters add accounting parentheses, row bands, and state classes.

Retrieve request

```
GET /api/v1/funding-panels/sec-corp?businessDate=2026-07-25
Accept: application/json
```

Representative response excerpt

```
{
  "schemaVersion": 1,
  "reportId": "sec-corp-2026-07-25",
  "panelCode": "sec-corp",
  "title": "Sec Corp",
  "businessDate": "2026-07-25",
  "currency": "USD",
  "timezone": "America/New_York",
  "asOf": "2026-07-25T13:42:18-04:00",
  "version": 17,
  "permissions": { "canEdit": true, "canSave": true },
  "periods": [
    { "id": "snapshot0830", "label": "8:30", "kind": "snapshot", "editable": true },
    { "id": "live", "label": "LIVE", "kind": "live", "editable": false }
  ],
  "rows": [
    {
      "id": "occ", "code": "OCC", "label": "OCC",
      "displayOrder": 30, "depth": 1,
      "kind": "detail", "valueMode": "input", "calculation": null,
      "values": {
        "snapshot0830": "-308824714.48",
        "snapshot1130": "-306000000.00",
        "snapshot1330": "-302500000.00",
        "live": "-300000000.00",
        "opportunityFunding": "-300000000.00"
      }
    }
  ]
}
```

4. Domain model: the business language

The domain model is the trusted internal representation. It does not know about AG Grid, CSS, HTTP status codes, or components.

Concept	Meaning
FundingReport	One complete panel for one business date and one version.
ReportPeriod	One of 8:30, 11:30, 1:30, LIVE, or Opps funding.
FundingRow	A stable row identity, label, hierarchy, kind, value mode, values, and optional calculation.
valueMode=input	User-editable only where the period and permissions also allow editing.
valueMode=calculated	Read-only result derived from other rows.
valueMode=section	A visual band with null numeric cells.
SaveFundingReportCommand	A full replacement of every editable snapshot value plus expectedVersion.

Exact money representation

Money is represented by branded canonical strings such as `-308824714.48`, then calculated with `decimal.js`. JSON numbers and ordinary JavaScript floating-point addition are avoided.

```
Valid:   "0.00", "1234.50", "-308824714.48"
Invalid: 1234.50, "1,234.50", "12.345", null on a numeric row
```

Why: JavaScript numbers use binary floating point. Simple-looking financial operations can produce values such as `0.30000000000000004`. Canonical strings preserve the wire format, while `decimal.js` performs predictable base-10 arithmetic.

Calculation graph

Calculated rows declare a sum of stable row IDs. The calculator resolves dependencies recursively, memoizes results, rejects missing rows and cycles, and recalculates all five periods. This supports immediate preview totals and validation of server totals.

```
"calculation": {
  "kind": "sum",
  "rowIds": ["occ", "nsc", "margin-fails-116-02149"]
}
```

Contract invariants

- The report version is a positive integer.
- The five required periods exist exactly once and have approved editability.
- Row IDs and display orders are unique.
- Section rows contain only null values; numeric rows contain all five values.
- Calculation operands exist, are not sections, and do not form a cycle.
- Calculated values supplied by the server reconcile with the declared graph.

5. DTOs, schemas, mapper, and gateways

TypeScript types disappear at runtime. Therefore, an HTTP response typed as FundingReport is not automatically safe. The application deliberately receives unknown, validates it with Zod, maps it to the domain, and then runs deeper business validation.

```
HTTP unknown
-> Zod fundingReportSchema
-> FundingReportDto
-> mapFundingReportDto
-> assertValidFundingReport
-> trusted FundingReport
```

Gateway abstraction

FundingPanelGateway exposes only getReport and putReport. The store depends on this interface through an Angular injection token, not on HttpClient.

Implementation	Use	Strengths	Limitations
HttpFundingPanelGateway	Integrated environments	Real REST calls, If-Match, runtime response validation	Needs deployed API, auth context, CORS/network handling
MockFundingPanelGateway	Local development and deterministic tests	No backend required; simulates versioning, totals, and conflicts	In-memory only; not a database or a full production simulation

Runtime selection

```
window.__FUNDING_PANEL_CONFIG__ = {
  apiBaseUrl: '/api',
  agGridEnterpriseLicenseKey: '<deployment-managed key>',
  businessDate: '2026-07-28',
  fundingPanelDataSource: 'http'
};
```

This keeps environment choices out of the store and components. The same compiled application can use mock data locally and HTTP after deployment.

Important: the mock is the default for self-contained development. A real environment must explicitly select http before the Angular bundle starts. The AG Grid license key is injected at runtime and must not be committed or logged.

6. The signals-first application store

FundingPanelStore is the coordinator. It owns the report lifecycle and exposes read-only signals to the UI. Each mounted panel provides its own store, so two docked panels cannot leak edits or conflicts into each other.

State	Purpose
serverReport	The last complete report confirmed by a successful GET or PUT.
edits	A sparse immutable overlay of committed values not yet confirmed by the server.
activeEdit	The current editor text plus its validation result.
query	The active panel code and business date used for refresh.
loadStatus / saveStatus	Drive loading, saving, error, saved, and conflict presentation.
conflict	Expected and current versions after a stale save.
refreshPending	Records a manual refresh that must wait for a valid save.

Computed state

- report overlays valid active/committed edits and recalculates totals.
- viewModel maps the preview report into presentation-only rows and cells.
- isDirty includes committed edits and a changed active editor value.
- hostState publishes canRefresh, isDirty, and saveStatus to the shell.

Why three representations?

If the UI directly mutated the server report, it would be difficult to distinguish what the server accepted from what the user merely typed. Keeping the confirmed report, sparse edits, and active editor separate makes retry, conflict handling, preview calculations, and dirty-state reporting explicit.

Effects are intentionally rare

Computed signals describe derived state. Effects are used only for imperative edges: publishing status to the host adapter and reacting to the host's refresh revision. Network calls remain explicit methods rather than being triggered by broad reactive effects, which makes request timing easier to reason about and test.

7. Edit, preview, validate, and autosave



Valid input changes computed totals before a network request is sent.
The server response becomes the new authoritative report and version.

On 409/412: keep local edits visible, stop autosave, and require confirmed reload.

Editor behavior

- Only input rows in 8:30, 11:30, and 1:30 can enter edit mode.
- LIVE and Opps funding are always read-only.
- canEdit and canSave permissions must both be true.
- Every keystroke is validated and a valid value immediately previews totals.
- Focus loss, Enter, or Tab commits; Escape cancels.
- Invalid input keeps the editor open and blocks save and refresh.

Accepted user input

The editor accepts familiar financial forms such as 1,234.5, \$1,234.50, (1,234.50), and -1234.50. It removes display punctuation and emits a signed two-decimal canonical value.

Why autosave on focus loss?

Approach	Advantages	Disadvantages
Autosave on commit (current)	Fast workflow; no forgotten Save button; aligns with spreadsheet behavior	More requests; requires strong error, queue, and concurrency handling
Explicit Save button	User controls batching; fewer requests	Larger unsaved sessions; easier to forget; complex revert UX
Debounced save while typing	Can feel instant	Unclear commit boundary; risky with invalid/partial financial input

The current choice uses a clear business event: finishing a valid cell edit. It avoids saving incomplete text while still removing the need for a global Save action.

8. Save serialization and persistence

PUT replaces the complete editable snapshot state. The client does not send only the single changed cell. This makes the saved version a coherent snapshot and makes retry behavior deterministic.

Save request

```
PUT /api/v1/funding-panels/sec-corp/sec-corp-2026-07-25
If-Match: "17"
Content-Type: application/json

{
  "schemaVersion": 1,
  "expectedVersion": 17,
  "businessDate": "2026-07-25",
  "snapshotValues": {
    "occ": {
      "snapshot0830": "-308824714.48",
      "snapshot1130": "-306000000.00",
      "snapshot1330": "-302500000.00"
    },
    "nsc": {
      "snapshot0830": "28012488.36",
      "snapshot1130": "28500000.00",
      "snapshot1330": "29000000.00"
    }
  }
}
```

The real request includes every input row. The shortened example shows only two rows.

Server transaction requirements

- Find the report using panel code, report ID, and business date.
- Atomically compare the stored version to expectedVersion/If-Match.
- Reject a stale request without applying any partial change.
- Replace the full editable snapshot state.
- Recalculate authoritative totals, update asOf, and increment version.
- Return the entire new report, not only the edited cell.

Queued save behavior

Only one PUT is allowed in flight per store. If the user edits another cell while a PUT is pending, the newer edit stays in the overlay. When the first response arrives, the store removes values that the server accepted and sends any remaining newer intent in the next PUT using the new version.

Why not parallel PUTs? Two requests with the same version would race. One would normally fail, and responses could arrive out of order. Serialization preserves a single understandable sequence.

9. Optimistic concurrency and failure handling

Optimistic concurrency assumes collisions are uncommon, so users do not lock the report while reading it. Every save proves which version the user edited.

Stale save example

```
User A loads version 17
User B loads version 17
User B saves -> server stores version 18
User A sends If-Match: "17" and expectedVersion: 17
Server rejects User A with 409 or 412

HTTP/1.1 409 Conflict
Content-Type: application/json

{
  "code": "VERSION_CONFLICT",
  "message": "The Sec Corp report was updated by another user.",
  "expectedVersion": 17,
  "currentVersion": 18
}
```

What the client does

- Maps HTTP 409 and 412 to FundingPanelVersionConflictError.
- Stops automatic saving.
- Keeps the user's attempted values visible and dirty.
- States that the screen is out of date and the changes were not saved.
- Blocks ordinary refresh so work is not silently erased.
- Requires a two-step confirmed reload before discarding local values.

There is intentionally no automatic merge. A generic client cannot know whether two changes to the same financial row are compatible. The safer current policy is to show the conflict, reload the authoritative report, and ask the user to re-enter intentional changes.

Other failure states

Condition	Behavior
GET failure	Show Try again; retain the active query for retry.
Transient PUT failure	Keep dirty values and offer Retry with the same version.
Invalid edit	Keep editor active; block save and refresh.
Refresh during save	Queue refresh; GET starts only after a clean successful save.
Duplicate refresh during GET	Reject the duplicate request.

10. View model and AG Grid

The domain model says what the report means. The grid view model says how each cell should behave on screen. Keeping them separate prevents presentation flags from polluting business data.

Cell view-model property	What the grid uses it for
value	Canonical amount or null for a section cell.
editable	Whether AG Grid may open the custom editor.
dirty	Whether to show the unsaved marker.
preview	Whether the cell is the active preview.
validationMessage	Whether to show invalid styling and accessible feedback.

Grid responsibilities

- Creates Bucket plus the five period columns from report metadata.
- Uses stable row IDs so updates do not replace unrelated rows.
- Keeps LIVE, Opps funding, sections, and calculated rows read-only.
- Uses the custom Angular amount editor only for snapshot columns.
- Formats negatives with parentheses without converting strings to numbers.
- Applies centralized semantic classes for sections, totals, dirty, and invalid cells.
- Provides screen-reader label and description through AG Grid's API.

Why AG Grid?

Option	Advantages	Disadvantages
AG Grid (current)	Mature grid behavior; keyboard editing, virtualization, enterprise path	Large dependency; license management; third-party DOM/API boundary
Native HTML table	Small and semantic; complete control	Substantial custom work for editing, focus, resize, scroll, and large datasets
Angular CDK table	Angular-native primitives and flexibility	Still requires custom spreadsheet editing and many grid features

Only the required AG Grid modules are registered. This reduces unused code while keeping the Enterprise license path available for future features.

11. Panel-specific behavior and reuse

Sec Corp

- Route: /sec-corp.
- 37 ordered target rows.
- Six placeholder labels were resolved from the supplied source mapping.
- Its component supplies the Sec Corp fixture and panel code.

PBIL

- Route: /pbil.
- 27 ordered target rows.
- Confirmed labels include Slab activity (2147), Prime reserve requirement, 15C3 Withdrawal, and PBIL EOD Balance.
- Provisional source allocations are recorded as metadata in the PBIL row catalog.
- LIVE and Opps funding currently use 0.00 where source values are unavailable.

How to add a third funding panel

Step	Change
1	Create panels// <code>-panel.component.*</code> .
2	Provide <code>FundingPanelStore</code> and the shared data-access providers.
3	Add the panel code and title to <code>FundingPanelSurfaceComponent</code> .
4	Create a stable ordered row catalog and document mapping assumptions.
5	Add a validated representative fixture for mock mode.
6	Add a lazy route and panel-specific contract/component tests.
7	Do not fork the shared grid/store unless business behavior truly differs.

Labels, order, values, permissions, and sum dependencies belong in report data. A shared behavior change belongs in the reusable infrastructure. This rule is the main defense against copy-and-paste panels.

12. Testing strategy

The suite contains 77 tests across 18 test files. Tests are layered so failures point to the smallest responsible area.

Test level	Examples
Pure domain	Canonical decimals, exact sums, dependency recalculation, malformed calculations.
Application store	Preview totals, invalid input, queued saves, retries, refresh, stale versions.
Data access	GET parsing, complete PUT shape, If-Match, 409/412 mapping, mock atomicity.
Presentation/component	Editable columns, formatting, row classes, editor keyboard behavior, accessibility.
Panel integration	Sec Corp/PBIL mounting, scoped stores, PBIL autosave and stale-update notice.
App routing	Both lazy panel routes mount successfully.

Important stale-version test

The PBIL component test loads version 7, simulates another user saving version 8 through the same mock persistence boundary, then attempts a local version-7 save. It asserts conflict state, retained dirty work, and the visible reload message. A mock gateway test separately proves the rejected PUT does not mutate server state.

Release-quality command

```
npm run check
npm audit --omit=dev
```

`npm run check` verifies Prettier formatting, ESLint, all unit/component tests, and the production Angular build. The production dependency audit currently reports zero vulnerabilities.

What these tests do not replace

- A real backend integration test proving atomic database compare-and-update.
- An authenticated deployment smoke test inside the enterprise shell.
- Visual regression testing across supported browsers and docked sizes.
- Load, latency, observability, and failure-injection testing of the deployed API.

13. Design decisions: benefits and costs

Decision	Why it helps	Cost or risk
Standalone Angular only	Clear imports, lazy routes, no NgModule indirection	Provider scope must be understood carefully
Signals-first store	Simple synchronous derived state and OnPush rendering	Async workflows still need explicit queue/state logic
Domain separate from grid VM	Business model stays reusable and testable	Requires a mapping layer and more types
Full-state PUT	Coherent snapshot, deterministic retry and server validation	Larger payload and higher collision scope than PATCH
Optimistic concurrency	No long-lived locks; scalable read/edit experience	Users can encounter conflicts and must reload
Exact decimal strings	Safe wire representation and base-10 calculations	More normalization and formatting code
Runtime Zod validation	Bad API data fails at the boundary with useful paths	Runtime validation and schema maintenance overhead
Manual refresh only	Predictable and avoids silently replacing work	Users may view older data until they refresh

Why this balance fits the current domain

Funding values require exact arithmetic, clear operator intent, and safe multi-user updates. The implementation spends extra code on explicit state and validation in exchange for predictable failure handling. That is appropriate for an enterprise financial panel, where a silent overwrite is more expensive than a visible conflict.

14. Future improvements

Near term

- Integrate with the real API and add contract tests against a deployed test service.
- Add a backend concurrency integration test that proves compare, update, total recalculation, version increment, and audit record occur in one transaction.
- Add visual regression tests for the reference screenshots and narrow docked sizes.
- Finalize the provisional PBIL source allocations and calculation rules.
- Add structured, payload-safe telemetry for GET/PUT latency, 5xx, and 409/412 counts.

Medium term

- Introduce a conflict comparison view showing server value versus local attempted value before the user reloads.
- Consider field-level merge only after business owners define safe merge semantics.
- Generate shared OpenAPI schemas or client types to reduce drift, while retaining runtime validation.
- Add a panel registry if the number of funding panels grows enough to make route and provider setup repetitive.
- Evaluate polling or server push only with explicit dirty/edit protection.

Possible PATCH alternative

PATCH could reduce payload size and conflict scope by sending only changed cells. However, it introduces operation ordering, partial-update validation, and more complex retry/merge rules. For the current report sizes, full-state PUT is simpler and safer. Reconsider PATCH only if payload size, latency, or collision frequency becomes a measured problem.

Possible server-driven row catalog

The REST report already carries row labels, order, value modes, and calculations. Panel catalogs currently document source mapping and protect fixture correctness. In the future, an authoritative backend configuration service could own more mapping metadata, provided it has versioning, review, and strong validation.

15. A practical debugging guide

Symptom	Where to look first
Panel does not load	Runtime config, selected gateway, GET URL, Zod/schema error, load status.
A row is missing or misplaced	Panel fixture/backend rows, displayOrder, row catalog, contract validation.
Cell cannot be edited	Period kind/editable, row valueMode, canEdit/canSave permissions.
Total is wrong during typing	Calculation rowIds, active overlay, decimal normalization, calculator test.
Edit stays dirty	PUT response value/version, save queue, rebaseEdits, network error.
Repeated conflict	Other writers, expectedVersion/If-Match, atomic backend version increment.
Mock data in production	window config timing and fundingPanelDataSource='http'.
AG Grid watermark	Runtime Enterprise license injection before panel initialization.

Suggested reading order for a junior developer

- Start with app.routes.ts and one thin panel component.
- Read funding-report.ts to learn the business vocabulary.
- Read funding-panel-surface.component.html to see user states.
- Read funding-panel.store.ts one use case at a time: load, edit, save, conflict.
- Read the gateway interface, then the mock gateway, then the HTTP gateway.
- Read the view-model mapper and grid component last.
- Run the nearest test after each file; the tests are executable documentation.

Mental model to keep: the server report is confirmed truth, local edits are intent, the computed report is a preview, the view model is presentation, and a successful versioned PUT is the only event that turns intent into confirmed truth.

16. Glossary

Term	Plain-language definition
Authoritative report	The latest complete report accepted and returned by the server.
Canonical decimal	A signed string with exactly two decimal places.
Computed signal	A read-only value automatically derived from other signals.
DTO	The JSON-shaped object crossing the network boundary.
Dirty	Changed locally but not yet confirmed by the server.
Gateway	An interface hiding whether data comes from HTTP or a mock.
Optimistic concurrency	Save only if the server is still at the version the user loaded.
Overlay	A sparse set of local changes applied over the confirmed report.
Rebase	Remove accepted edits and keep only newer local intent after a PUT response.
Runtime validation	Checking actual JSON while the program runs, not only at compile time.
Standalone component	An Angular component that declares its own imports without an NgModule.
View model	A UI-specific shape derived from the domain model.

Final takeaway

This application is not two unrelated grids. It is one reusable, version-aware funding panel platform with two panel configurations. Its architecture protects financial accuracy, keeps UI concerns separate from business rules, prevents silent concurrent overwrites, and gives future panels a controlled path for reuse.

Repository release command: `npm run check`. Production persistence still depends on the backend honoring the documented atomic version transaction.